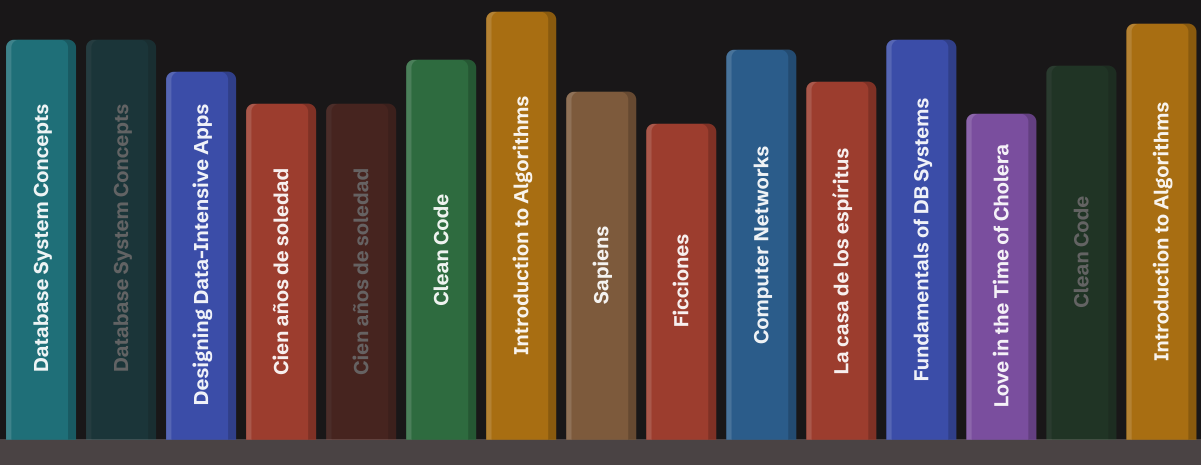
Laboratorio 01 · Bases de Datos

Biblioteca Universitaria, explicada desde cero.

Una guía para entender, sin saber nada de programación, qué hace cada parte del proyecto, cómo se conectan los archivos entre sí y por qué se construyó así.

Aplicación: Python + Flask + MySQL
Código: github.com/lcorb10/Biblioteca
En línea: biblioteca-93xx.onrender.com
Autor: Sebastián Corbacho · Universidad del Norte · Octubre 2026



CONTENIDO

Qué hay en esta guía

Puedes leerla de corrido o saltar a lo que necesites. Las partes 1 a 3 son la base: si las entiendes, todo lo demás encaja solo.

PARTE 1
El problema: qué pidió el profesor y qué construimos

PARTE 2
Conceptos básicos, explicados como a un niño

PARTE 3
El mapa: cómo se conectan todas las piezas

PARTE 4
El viaje de un clic, paso a paso

PARTE 5
La base de datos: tablas, relaciones y diagramas

PARTE 6
Archivo por archivo: qué hace y con quién habla

PARTE 7
Recorrido por la aplicación (con capturas)

PARTE 8
Las reglas del negocio: préstamos, multas y validaciones

PARTE 9
Seguridad: cómo se protegen los datos

PARTE 10
Cómo ejecutarla en tu computador

PARTE 11
Cómo está publicada en internet

PARTE 12
Preparación para la sustentación

ANEXO
Glosario y ficha rápida

Cómo leer los recuadros

En simple explica una idea con una comparación de la vida diaria. ¿Por qué? cuenta la razón detrás de una decisión. Ojo marca algo importante que no debes olvidar. Consejo son ayudas para la sustentación.

PARTE 1

El problema

Antes de ver código, entendamos qué teníamos que resolver.

Lo que pidió el profesor

La biblioteca de la universidad presta libros a mano (o con herramientas sueltas). El laboratorio pide construir una **aplicación web** que use una **base de datos relacional** para digitalizar el proceso de préstamo y devolución. En concreto:

Área	Qué debe permitir
Catálogo	Guardar libros (ISBN, título, autores, editorial, año, categorías, descripción) y sus ejemplares físicos (código de barras, ubicación, estado: disponible, prestado o perdido). Buscar por título, autor, ISBN, categoría y disponibilidad.
Usuarios	Registrar lectores (código, identificación, nombres, correo, rol estudiante/docente, carrera opcional, estado activo/bloqueado) y ver su historial de préstamos.
Préstamos	Registrar usuario + ejemplar + fecha de préstamo + fecha de vencimiento, validando antes que el usuario esté activo y el ejemplar disponible.
Devoluciones	Registrar la devolución, dejar el ejemplar disponible, calcular la multa si hay retraso y anotar cómo llegó el libro (dañado, páginas faltantes).
Reportes	Préstamos activos y vencidos, libros más prestados por periodo, usuarios con más préstamos e inventario por estado.
Reglas	Un libro tiene varios ejemplares y varios autores; un usuario puede tener varios préstamos; hay una tarifa de multa por día; CRUD de libros, ejemplares y usuarios; un bibliotecario aprueba préstamos y devoluciones; el usuario busca el libro y solicita el préstamo.

Además había que entregar: el **diagrama entidad-relación (MER)**, el **diagrama relacional**, el **script SQL** que crea las tablas con datos básicos y los **archivos de la aplicación**. El enunciado permitía Python (con Flask o Django) o JavaScript (Node + Express), y varios motores de base de datos; elegimos **Python + Flask + MySQL**.

En simple

Imagina el cuaderno donde la bibliotecaria anota quién se llevó qué libro. Nuestra aplicación es ese cuaderno, pero digital: muchas personas lo pueden consultar a la vez desde el navegador, no se pierden las hojas, y el sistema hace las cuentas solo (por ejemplo, cuánto debe alguien que devolvió tarde).

Lo que construimos

1

Para el lector (estudiante o docente)

Crea su cuenta, busca libros en el catálogo (con portadas reales), solicita un préstamo, ve sus libros, sus vencimientos y sus multas, e imprime comprobantes.

2

Para el bibliotecario

Administra libros, ejemplares y usuarios; aprueba o rechaza solicitudes; presta directo en el mostrador; registra devoluciones con multa automática; consulta reportes y exporta a Excel.

3

Base de datos sólida

12 tablas con claves primarias, foráneas, restricciones UNIQUE y ENUM, y transacciones que impiden prestar dos veces el mismo ejemplar.

4

Extras para la sustentación

Botón **Ver SQL** que muestra las consultas reales de cada pantalla, modo oscuro, versión para celular, animaciones y publicación en internet.

PARTE 2

Conceptos básicos

Estas son las palabras que vas a escuchar todo el tiempo. Cada una con su comparación de la vida real.

Cómo funciona una página web

Navegador (cliente)

Chrome, Edge o Safari. Es quien **pide** páginas y las muestra. Es el cliente en un restaurante.

Servidor

Un programa que está esperando pedidos y responde con páginas. En nuestro caso es **Flask**. Es el mesero.

Base de datos

Donde se guarda la información de forma ordenada y permanente. Es la despensa del restaurante.

Cada vez que haces clic, el navegador envía una **petición** (en inglés *request*) a una dirección como `/prestamos`. El servidor la recibe, consulta la base de datos si lo necesita, arma una página HTML y la devuelve como **respuesta**. Hay dos tipos de peticiones que usamos:

- **GET**: "muéstrame algo" (ver el catálogo, abrir un reporte). No cambia datos.
- **POST**: "haz algo" (aprobar un préstamo, guardar un libro). Sí cambia datos.

La base de datos relacional

En simple

Una base de datos relacional es como un libro de Excel con varias hojas, donde cada hoja es una **tabla**. La hoja "usuario" tiene una fila por persona; la hoja "libro", una fila por título. Lo especial es que las hojas están **conectadas** entre sí por números de referencia, y el motor (MySQL) vigila que esas conexiones nunca se rompan.

Concepto	Qué es	Ejemplo en el proyecto
Tabla	Un grupo de datos del mismo tipo.	`usuario`, `libro`, `prestamo`
Fila (registro)	Un elemento concreto.	Laura Pérez es una fila de `usuario`.
Columna (campo)	Un dato que tienen todas las filas.	`correo`, `estado`
Clave primaria (PK)	El "número de cédula" de cada fila: único e irrepetible. Identifica la fila.	`id_usuario`, `isbn`
Clave foránea (FK)	Una columna que guarda la PK de otra tabla. Es el "hilo" que conecta tablas. MySQL no deja apuntar a algo que no existe.	`prestamo.id_usuario` apunta a `usuario.id_usuario`.

UNIQUE (UK)	Prohíbe valores repetidos en una columna, aunque no sea la PK.	Dos usuarios no pueden tener el mismo correo.
ENUM	Una lista cerrada de valores permitidos.	`estado` del ejemplar: disponible, prestado o perdido.
NULL	"Vacío, sin dato todavía".	La carrera de un docente; el ejemplar de una solicitud aún no aprobada.

Las relaciones: 1 a N y N a M

Uno a muchos (1:N)

Un **libro** tiene muchos **ejemplares**, pero cada ejemplar es de un solo libro. Se resuelve poniendo la FK en el lado "muchos": `ejemplar.isbn`.

Muchos a muchos (N:M)

Un libro tiene varios **autores** y un autor escribe varios libros. Esto no cabe en una sola columna, así que se crea una **tabla puente**: `libro_autor`, con una fila por cada pareja libro–autor.

SQL: el idioma de la base de datos

SQL es el lenguaje con el que se le habla a MySQL. Con cuatro verbos se hace casi todo (por eso se habla de **CRUD**: Crear, Leer, Actualizar, Borrar):

```sql
-- Leer (Read): ¿qué ejemplares de Clean Code están disponibles?
SELECT codigo_barras, ubicacion FROM ejemplar
WHERE isbn = '9780132350884' AND estado = 'disponible';

-- Crear (Create): registrar un usuario nuevo
INSERT INTO usuario (codigo, identificacion, nombres, correo, rol)
VALUES ('U009', '1009009009', 'Ana Gil', 'ana@uni.edu.co', 'estudiante');

-- Actualizar (Update): marcar un ejemplar como prestado
UPDATE ejemplar SET estado = 'prestado' WHERE id_ejemplar = 6;

-- Borrar (Delete)
DELETE FROM usuario WHERE codigo = 'U009';
```

Transacción: todo o nada

En simple

Cuando se aprueba un préstamo hay que hacer dos cosas: marcar el ejemplar como "prestado" y marcar la solicitud como "activa". Si se hiciera una y fallara la otra, quedaría un libro "prestado" que nadie tiene. Una **transacción** agrupa varios pasos para que se guarden **todos juntos** (`COMMIT`) o **ninguno** (`ROLLBACK`). Es como una transferencia bancaria: o sale de una cuenta y llega a la otra, o no pasa nada.

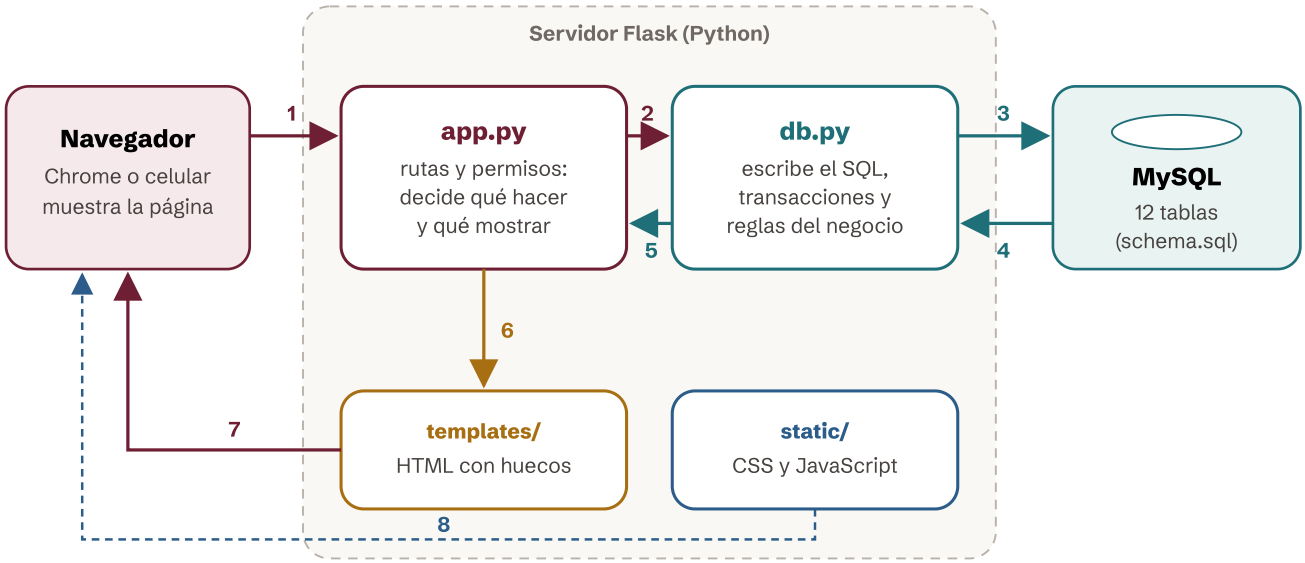
Las herramientas del proyecto

Herramienta	Qué es y para qué la usamos
Python	El lenguaje de programación del servidor. Es conocido por ser fácil de leer.
Flask	Un *framework* (caja de herramientas) de Python para crear sitios web. Se encarga de recibir peticiones y decidir qué función responde a cada dirección.
MySQL 8	El motor de base de datos relacional. Guarda las tablas y ejecuta el SQL.
PyMySQL	El "traductor" (driver) que permite a Python conectarse y enviar SQL a MySQL.
Jinja2	El sistema de plantillas de Flask: HTML con huecos que se llenan con datos (por ejemplo `{{ libro.titulo }}`).
HTML, CSS y JavaScript	HTML es la estructura de la página (los ladrillos), CSS el diseño (la pintura) y JavaScript el comportamiento en el navegador (los interruptores).
Entorno virtual (.venv)	Una carpeta con las librerías del proyecto aisladas, para que no choquen con otros proyectos del computador.
Git y GitHub	Git guarda el historial de cambios del código; GitHub es la página donde se guarda en internet (github.com/lcorb10/Biblioteca).
Render y Aiven	Servicios en la nube: Render ejecuta la aplicación Flask y Aiven aloja la base de datos MySQL. Así la app funciona en internet sin tu computador encendido.

PARTE 3

El mapa

Este es el dibujo más importante de la guía: cómo viaja la información entre las piezas.



#	Qué pasa
1	El navegador envía la **petición** (por ejemplo, abrir `/prestamos` o aprobar una solicitud).
2	`app.py` revisa quién eres y le pide los datos a `db.py`.
3	`db.py` envía el **SQL** a MySQL.
4 · 5	MySQL devuelve las **filas** y `db.py` se las entrega a `app.py`.
6	`app.py` llena una **plantilla** HTML con esos datos.
7	La página terminada vuelve al navegador como **respuesta**.
8	El navegador descarga el diseño (CSS) y el comportamiento (JavaScript) de `static/`.

En simple: el restaurante

El **navegador** es el cliente que pide un plato. **app.py** es el mesero: toma el pedido, revisa que el cliente pueda pedirlo y se lo pasa a la cocina. **db.py** es el cocinero: sabe las recetas (el SQL) y va a la **despensa** (MySQL) por los ingredientes. Las **plantillas** son la vajilla donde se sirve el plato, y **static/** es la decoración del restaurante.

Las tres capas

Separamos el proyecto en capas para que cada archivo tenga **una sola responsabilidad**. Así es más fácil de entender, de explicar y de modificar.

Capa	Archivos	Responsabilidad
Presentación	`templates/*.html`, `static/css/app.css`, `static/js/app.js`	Cómo se ve y cómo reacciona la página. No sabe nada de SQL.
Lógica / control	`app.py`	Rutas (qué URL hace qué), permisos (quién puede entrar) y mensajes al usuario.
Datos	`db.py`, `schema.sql`	Toda la comunicación con MySQL, las consultas y las reglas del negocio (validar, calcular multas).

¿Por qué separarlo así?

Si mañana el profesor pide cambiar el diseño, solo se tocan las plantillas y el CSS. Si pide cambiar una regla (por ejemplo, 10 días de préstamo en vez de 7), se toca `db.py` o la tabla `configuracion`. Ninguna capa se entera de los cambios de las otras.

PARTE 4

El viaje de un clic

Sigamos un caso real de principio a fin: el bibliotecario aprueba la solicitud de María para llevarse *Clean Code*.

1. **El clic.** En la pantalla Préstamos, el bibliotecario elige el ejemplar `EJ-0006`, la fecha de vencimiento y pulsa **Aprobar préstamo**. El formulario HTML (en `templates/prestamos.html`) envía un **POST** a `/prestamos/3/aprobar`, donde 3 es el número de la solicitud.

2. **Flask encuentra la función.** En `app.py` hay una línea que dice "esta URL la atiende esta función":

```python
@app.post("/prestamos/<int:id_prestamo>/aprobar")
@requiere("personal")          # solo el personal de la biblioteca puede entrar
def prestamo_aprobar(id_prestamo):
    venc = parse_fecha(request.form.get("vence"))
    db.aprobar_solicitud(id_prestamo, int(request.form["id_ejemplar"]), auth()["id"], venc)
    flash("Préstamo aprobado. El ejemplar quedó marcado como prestado.", "ok")
    return redirect(url_for("prestamos"))
```

3. **Se revisan los permisos.** El "decorador" `@requiere("personal")` mira la sesión: si quien hace clic no inició sesión como bibliotecario, lo saca (error 403). Un lector nunca puede aprobar préstamos.

4. **db.py abre una transacción y valida.** La función `_validar_y_prestar` hace, en orden:

```sql
START TRANSACTION;
SELECT estado FROM usuario WHERE id_usuario=3 FOR UPDATE;        -- ¿María está activa?
SELECT estado, isbn FROM ejemplar WHERE id_ejemplar=6 FOR UPDATE;  -- ¿el ejemplar está
disponible?
UPDATE ejemplar SET estado='prestado' WHERE id_ejemplar=6;
UPDATE prestamo SET id_ejemplar=6, id_personal=1, fecha_prestamo='2026-10-19',
       fecha_vencimiento='2026-10-26', estado='activo'
 WHERE id_prestamo=3 AND estado='solicitado';
COMMIT;
```

   Si alguna validación falla (usuario bloqueado, ejemplar ya prestado, fecha inválida), se hace `ROLLBACK` y no se guarda nada.

5. **FOR UPDATE: el candado.** Esas dos palabras "bloquean" las filas mientras dura la transacción. Si dos bibliotecarios intentan prestar el mismo ejemplar al mismo tiempo, el segundo espera, ve que ya está prestado y recibe un error. Nunca se presta dos veces.

6. **Mensaje y redirección.** `flash( … )` guarda el mensaje "Préstamo aprobado" y `redirect` le dice al navegador "ahora ve a /prestamos".

7. **Se arma la página nueva.** El navegador pide `/prestamos` (GET). La función `prestamos()` consulta las solicitudes y los préstamos activos con `db.py` y se los pasa a la plantilla `prestamos.html`, que los convierte en filas de tabla.

8. **El navegador muestra el resultado.** Aparece el aviso verde animado y la solicitud ya no está en la lista. Si pulsas **Ver SQL**, verás exactamente las sentencias del paso 4.

Consejo para la sustentación

Este recorrido es la mejor forma de demostrar que entiendes el proyecto. Hazlo en vivo: aprueba una solicitud y abre el panel **Ver SQL** para mostrar la transacción con su `COMMIT`.

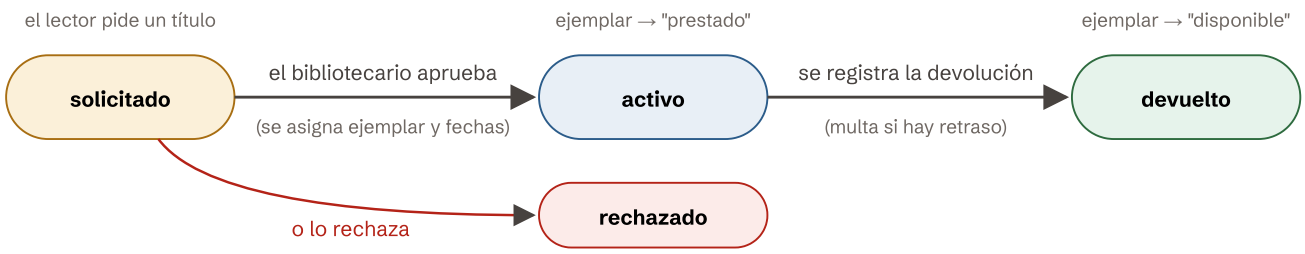
PARTE 5

La base de datos

Doce tablas, cada una con un propósito claro. Todas se crean con el archivo `schema.sql`.

Tabla	Qué guarda	Datos clave
`libro`	Cada título del catálogo (la "obra", no la copia física).	PK `isbn`; FK opcional a editorial
`ejemplar`	Cada copia física que se puede prestar.	PK `id_ejemplar`; `codigo_barras` único; estado ENUM
`autor`	Los autores, una sola vez cada uno.	`nombre` único
`libro_autor`	Puente N:M: qué autor escribió qué libro.	PK compuesta (isbn, id_autor)
`categoria`	Temas: Bases de datos, Literatura, Redes…	`nombre` único
`libro_categoria`	Puente N:M: en qué categorías está cada libro.	PK compuesta (isbn, id_categoria)
`editorial`	Las editoriales.	`nombre` único
`usuario`	Lectores: estudiantes y docentes.	código, identificación y correo únicos; estado activo/bloqueado
`personal`	Bibliotecarios y administradores que aprueban y reciben.	`login` único; clave guardada como hash
`prestamo`	Cada solicitud y préstamo, de principio a fin.	FK a usuario, libro, ejemplar y personal; estado ENUM
`devolucion`	El cierre de un préstamo: fecha, retraso, multa y condición.	FK única a préstamo (relación 1 a 0..1)
`configuracion`	Parámetros del negocio que se pueden cambiar sin tocar código.	tarifa de multa, días de préstamo, máximo de préstamos

La vida de un préstamo

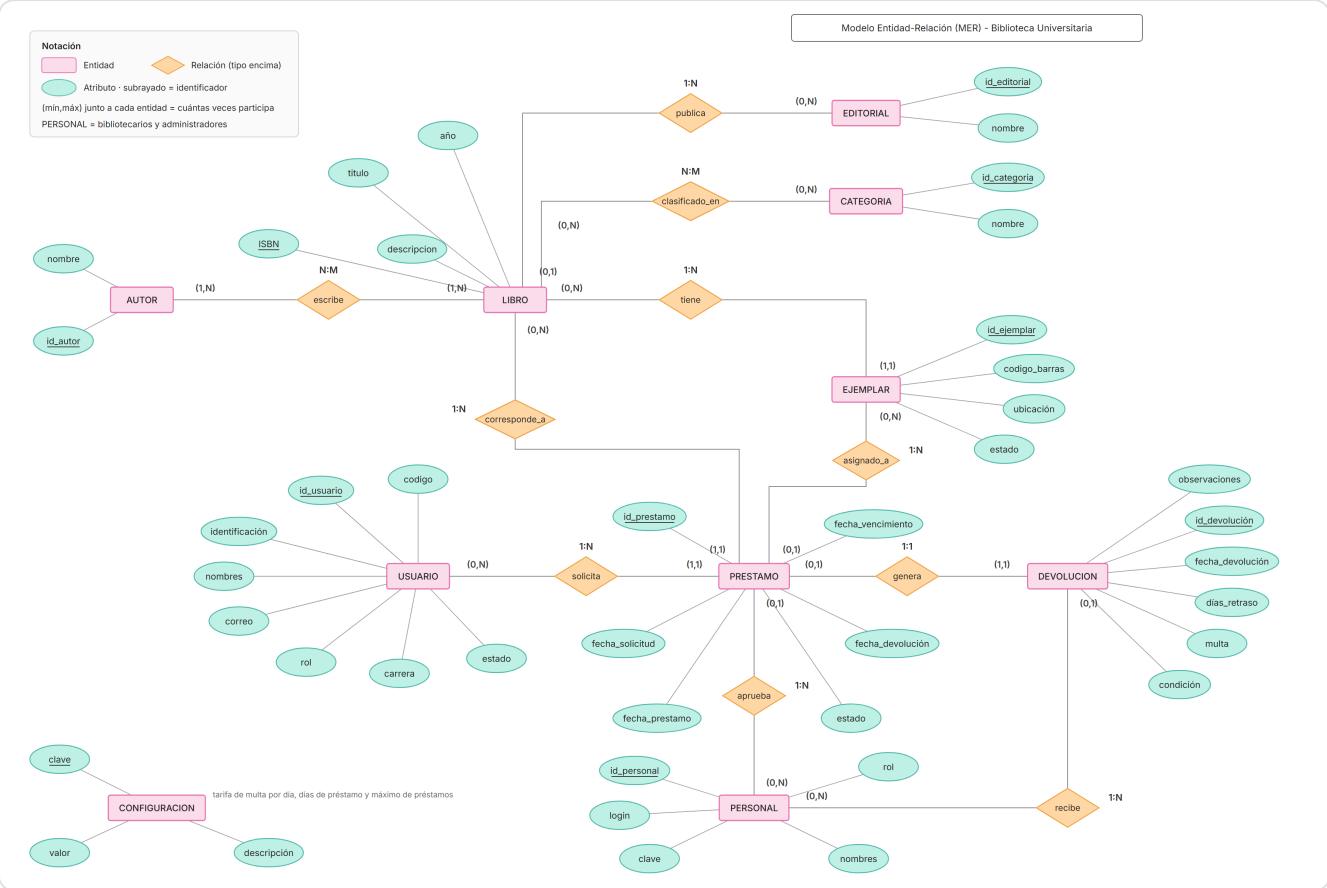


¿Por qué el préstamo guarda el ISBN y además el ejemplar?

Porque el lector pide un **título** ("quiero Clean Code"), no una copia concreta. La copia la escoge el bibliotecario al aprobar. Por eso `prestamo.isbn` es obligatorio desde el inicio y `prestamo.id_ejemplar` empieza vacío (NULL) hasta la aprobación. El préstamo directo en el mostrador llena ambos de una vez.

Diagrama entidad-relación (MER)

El MER es el dibujo **conceptual**: muestra las cosas del mundo real (entidades), sus características (atributos) y cómo se relacionan, sin pensar todavía en tablas. Está en notación de Chen: rectángulos = entidades, rombos = relaciones, óvalos = atributos (subrayado = identificador), y junto a cada entidad la cardinalidad **(mín, máx)**.

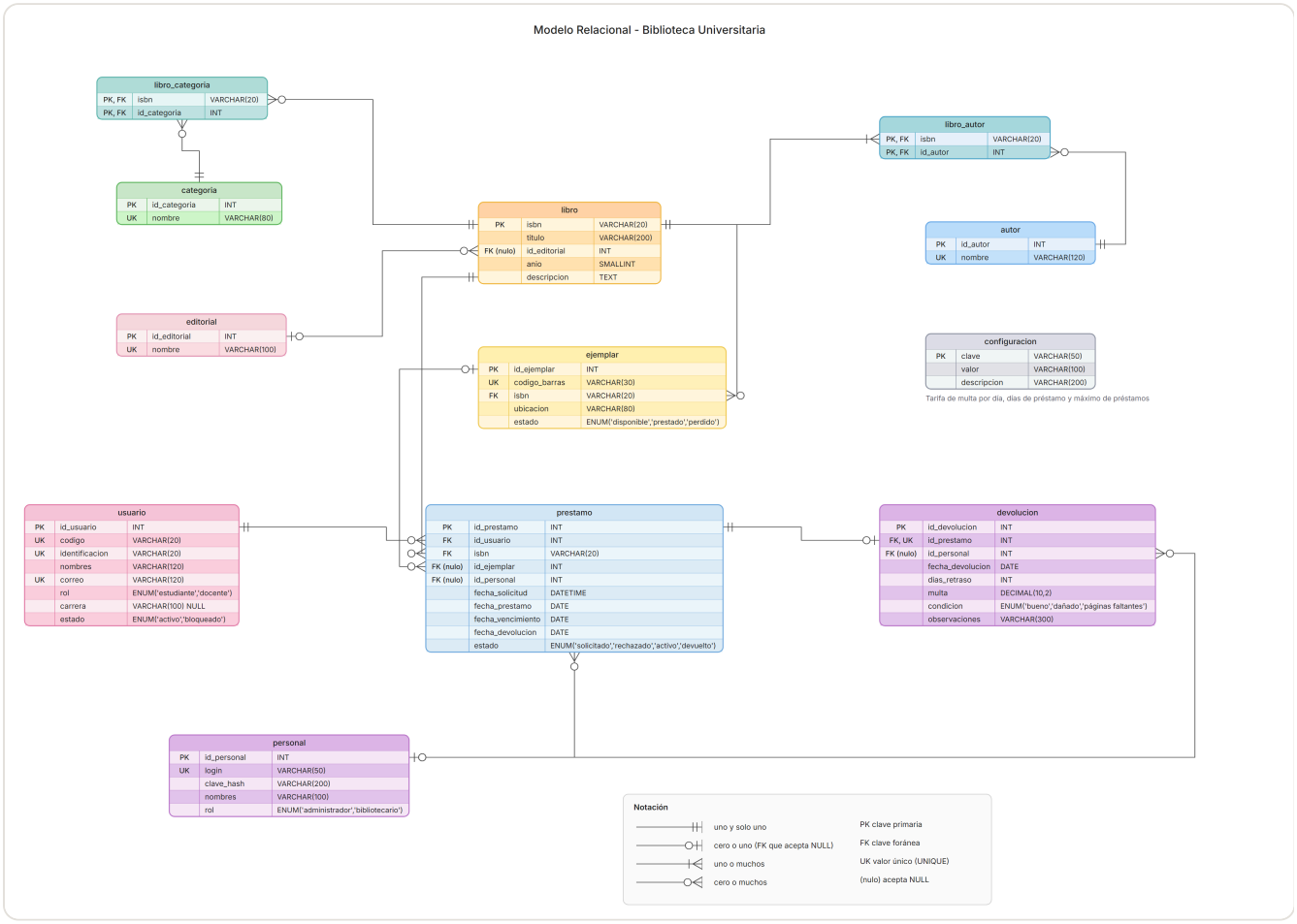


Cómo leer una cardinalidad

En la relación *solicita*, junto a USUARIO dice **(0,N)**: un usuario puede tener desde 0 hasta muchos préstamos. Junto a PRESTAMO dice **(1,1)**: cada préstamo pertenece a exactamente un usuario. Encima del rombo, **1:N** resume el tipo de relación.

Diagrama relacional

El modelo relacional es el MER convertido en **tablas reales**: aparecen las columnas con su tipo de dato, las claves (PK, FK, UK) y las tablas puente que resuelven los N:M. Las líneas usan notación de "patas de gallo": la raya es "uno", el tridente es "muchos" y el círculo significa "puede ser cero".



PARTE 6

Archivo por archivo

Qué hace cada archivo, con quién habla y por qué existe. Debajo de cada uno, la línea "Se conecta con" te dice sus vecinos en el mapa.

El corazón del proyecto

`app.py` Python · 740 líneas

El **controlador**. Crea la aplicación Flask y define las **29 rutas**: cada URL (`/catalogo` , `/prestamos/3/aprobar` ...) apunta a una función que recibe la petición, llama a `db.py` y elige qué plantilla mostrar. También contiene:

- **Permisos:** el decorador `requiere()` exige sesión, y `requiere("personal")` o `requiere("usuario")` limitan cada pantalla según el rol.
- **Filtros de plantilla:** pequeñas funciones que dan formato: `fecha` (19 oct 2026), `dinero` ($10.000), `color` (el color del lomo según la categoría), `iniciales` , `hace` ("hace 2 h").
- **Panel Ver SQL:** guarda cada sentencia ejecutada durante la petición para mostrarla en pantalla.
- **Una conexión por petición:** abre una sola conexión a MySQL por página y la cierra al terminar (más rápido, sobre todo en la nube).
- **Exportar CSV** y **manejo de errores** (páginas amables para "sin conexión", "sin permiso" o "no encontrado").

Se conecta con: **db.py** (le pide los datos) · **templates/** (les pasa los datos) · el navegador (recibe peticiones).

`db.py` Python · 610 líneas

La **capa de datos**. Es el único archivo que habla con MySQL. Tiene tres partes:

- **Conexión:** `load_config()` lee las credenciales (de `config.toml` en tu PC o de variables de entorno en la nube); `get_conn()` abre la conexión; `query()` ejecuta lecturas y `run_tx()` ejecuta transacciones con COMMIT/ROLLBACK.
- **Consultas:** una función por necesidad, con nombres en español: `buscar_libros` , `listar_usuarios` , `prestamos_activos` , `libros_mas_prestados` , `prestamos_por_mes` ...
- **Reglas del negocio:** `solicitar_prestamo` (valida usuario activo, máximo de préstamos y disponibilidad), `_validar_y_prestar` (la transacción de la parte 4), `registrar_devolucion` (calcula retraso y multa), `hash_pwd` / `check_pwd` (claves seguras).

Se conecta con: **MySQL** (envía SQL) · **app.py** (le responde con los datos) · **config.toml** o variables de entorno (credenciales).

`schema.sql` SQL · entregable

El **plano de la base de datos**. Borra y vuelve a crear la base `biblioteca`, crea las 12 tablas con sus PK, FK, UNIQUE, ENUM e índices, e inserta datos de ejemplo: 11 libros, 19 ejemplares, 8 usuarios, el administrador y 37 préstamos (34 ya devueltos en los últimos meses, para que los reportes tengan qué mostrar).

Se conecta con: **MySQL** (se ejecuta allí) · **cargar_bd.py** (lo ejecuta automáticamente) · los diagramas en **docs/** (lo describen).

`cargar_bd.py` Python · utilidad

Un ayudante que ejecuta `schema.sql` completo con un solo comando. `python cargar_bd.py` carga tu MySQL local; `python cargar_bd.py config.nube.toml` carga la base en Aiven. Al final imprime cuántas filas quedaron en cada tabla.

Se conecta con: **schema.sql** (lo lee) · **config.toml / config.nube.toml** (a qué servidor conectarse).

Configuración y despliegue

`requirements.txt`

La lista de librerías que necesita el proyecto: `flask`, `pymysql`, `cryptography` (para el cifrado de MySQL 8) y `gunicorn` (el servidor web que usa Render). `pip install -r requirements.txt` las instala todas.

`config.toml` no se sube

Las credenciales de tu MySQL local (host, usuario, clave, base) y la clave secreta de Flask. Está en `.gitignore` para que tu contraseña nunca llegue a GitHub.

`config.example.toml`

La plantilla pública de `config.toml`, con `TU_CLAVE` en lugar de la contraseña real. Quien descargue el proyecto la copia y la llena.

`config.nube.toml` no se sube

Los datos de la base en Aiven, para cargarla desde tu PC con `cargar_bd.py`. También está fuera de Git.

`render.yaml`

Las instrucciones para Render: qué instalar, cómo arrancar (`gunicorn app:app`) y qué variables de entorno necesita (`MYSQL_HOST`, `MYSQL_PASSWORD` ...).

`.gitignore`

La lista de lo que Git **no** debe subir: los archivos de credenciales, el entorno virtual y archivos temporales.

Las plantillas (templates/)

Cada pantalla es un archivo HTML con "huecos" de Jinja2. Todas heredan de `base.html`, que pone el menú lateral, los avisos, el buscador Ctrl+K y el panel Ver SQL; así no se repite ese código en cada página.

Archivo	Pantalla	Ruta(s) en app.py
`base.html`	Esqueleto común: menú, avisos, buscador, modo oscuro, panel SQL, animación de bienvenida.	todas
`_macros.html`	Piezas reutilizables: el estante de lomos, mini portadas, etiquetas de estado, botón eliminar.	(se importa)
`_historial.html`	Línea de tiempo de préstamos de un usuario.	historial, mis libros
`login.html` / `registro.html`	Ingreso (lector o bibliotecario) y creación de cuenta.	`/login`, `/registro`
`inicio.html`	Tablero del bibliotecario: contadores, estante, vencidos y solicitudes.	`/`
`catalogo.html` / `libro_detalle.html`	Búsqueda con portadas y página de cada libro.	`/catalogo`, `/libros/<isbn>`
`libros.html` / `libro_form.html`	CRUD de libros (lista, crear, editar, eliminar).	`/libros…`
`ejemplares.html` / `ejemplar_form.html`	CRUD de ejemplares.	`/ejemplares…`
`usuarios.html` / `usuario_form.html` / `historial.html`	CRUD de usuarios e historial de cada uno.	`/usuarios…`
`prestamos.html` / `prestamo_nuevo.html`	Aprobar o rechazar solicitudes, préstamos activos y préstamo directo.	`/prestamos…`
`devoluciones.html`	Registrar devoluciones con la multa calculada en vivo.	`/devoluciones`
`comprobante.html`	Recibo imprimible de préstamo o devolución.	`/prestamos/<id>/comprobante`
`reportes.html`	Los cuatro reportes, gráficos y exportación CSV.	`/reportes`, `/reportes/<tipo>.csv`
`mis_prestamos.html`	Los libros del lector que inició sesión.	`/mis-prestamos`
`error.html`	Páginas de error amables.	(errores 403, 404, 503)

El diseño (static/)

`static/css/app.css`

Todo el aspecto visual: colores (vino tinto como acento), tipografía Schibsted Grotesk, tarjetas, tablas, formularios, el estante de lomos, las portadas, el modo oscuro, la versión para celular, el comprobante y todas las animaciones. Usa **variables** (`--wine`, `--bg`...), así el modo oscuro solo cambia los valores de esas variables.

`static/js/app.js`

El comportamiento en el navegador: el buscador Ctrl+K, la multa que se recalcula al cambiar la fecha, la confirmación antes de eliminar, los avisos que desaparecen solos, los contadores animados, el filtro instantáneo de tablas, el cambio de tema, el panel SQL con colores y la animación de bienvenida.

¿Por qué JavaScript si ya hay Python?

Python corre en el servidor y solo actúa cuando llega una petición. JavaScript corre en el navegador y reacciona al instante sin recargar la página: por ejemplo, mostrar la multa mientras cambias la fecha. Las reglas importantes (la multa real, las validaciones) siempre se repiten en el servidor; lo del navegador es solo una vista previa.

Documentación y otros

Archivo o carpeta	Para qué sirve
`README.md`	La portada del repositorio en GitHub: cómo ejecutar el proyecto, cuentas de prueba y la tabla de cumplimiento del enunciado.
`modelo.md`	Los diagramas MER y relacional con su explicación y el modelo relacional en texto.
`docs/MER.png`, `docs/Modelo_Relacional.png`	Los diagramas entregables. Sus fuentes editables están en `docs/fuentes/` y hay versiones en Mermaid (`.mmd`).
`legacy_streamlit/`	La primera versión de la app hecha en Streamlit. Se guarda solo como referencia; ya no se usa porque el enunciado pedía Flask o Django.
`design-demos/`	Las tres propuestas de diseño iniciales (Ficha, Mostrador y Estante). Se eligió **Estante**.
`.venv/`	El entorno virtual con las librerías instaladas en tu PC. No se sube a GitHub. La carpeta `venv/` vieja era de otro computador y no sirve.

Quién llama a quién

Esta tabla resume las conexiones más importantes: qué ruta usa qué función de datos, qué tablas toca y qué plantilla muestra.

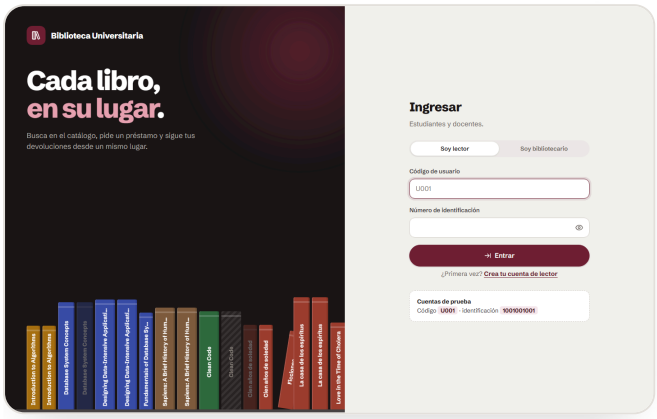
Acción	Ruta (app.py)	Función (db.py)	Tablas	Plantilla
Iniciar sesión	`login`	`autenticar_personal / _usuario`	personal, usuario	login.html
Crear cuenta	`registro`	`crear_usuario`	usuario	registro.html
Buscar libros	`catalogo`	`buscar_libros`	libro, autor, categoria, ejemplar…	catalogo.html
Solicitar préstamo	`solicitar`	`solicitar_prestamo`	usuario, prestamo, ejemplar	(redirige)
Aprobar	`prestamo_aprobar`	`aprobar_solicitud`	usuario, ejemplar, prestamo	(redirige)
Devolver	`devoluciones`	`registrar_devolucion`	prestamo, devolucion, ejemplar, configuracion	devoluciones.html
Reportes	`reportes`	`prestamos_activos`, `libros_mas_prestados` …	casi todas	reportes.html
CRUD libros	`libro_nuevo / editar / eliminar`	`guardar_libro`, `eliminar_libro`	libro, autor, categoria, editorial y puentes	libro_form.html

PARTE 7

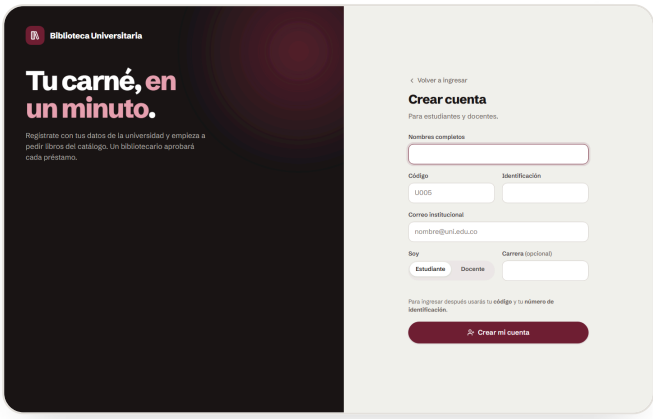
Recorrido por la aplicación

Así se ve cada pantalla y qué puedes hacer en ella.

Ingreso y registro

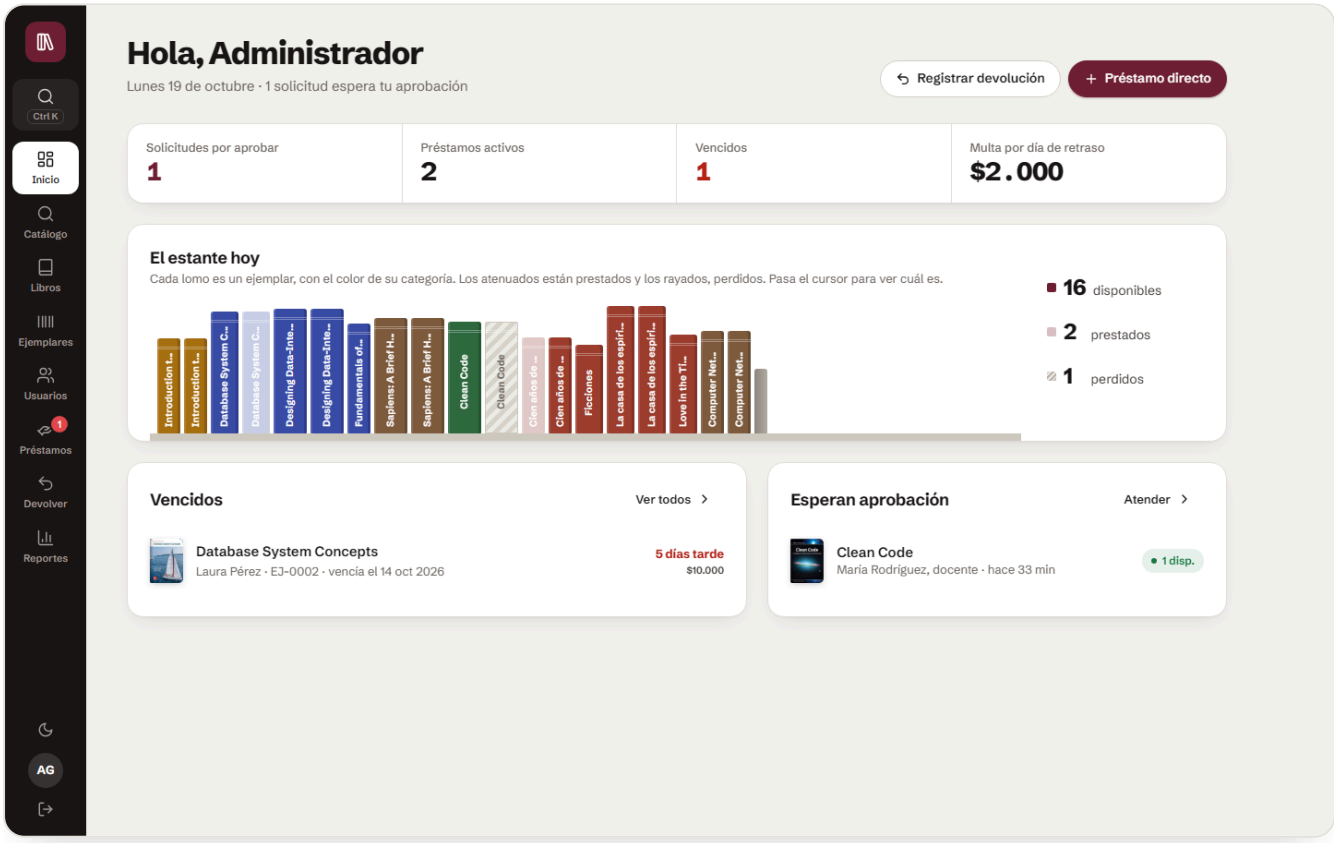


Login con dos modos: "Soy lector" (código + identificación) y "Soy bibliotecario" (usuario + clave). La repisa de la izquierda se dibuja con los ejemplares reales.



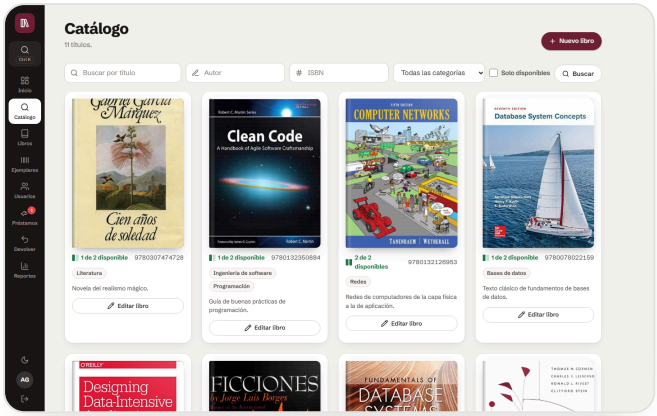
Registro de lectores con los datos mínimos que pide el enunciado. Al crear la cuenta entra directo y aparece la animación de bienvenida.

Inicio del bibliotecario

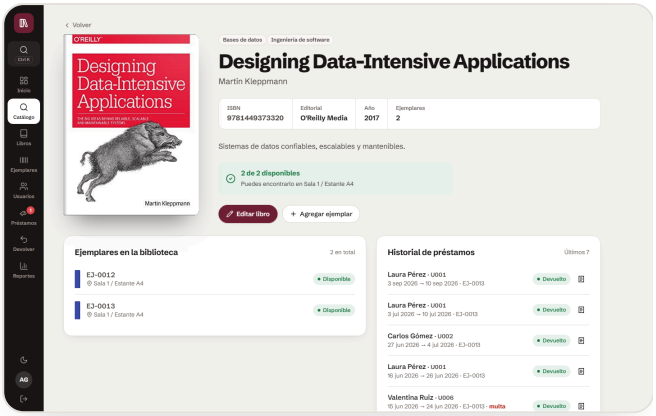


Contadores (solicitudes, activos, vencidos, tarifa), el **estante** donde cada lomo es un ejemplar (color = categoría, atenuado = prestado, rayado = perdido) y las listas de vencidos y solicitudes pendientes.

Catálogo y detalle del libro

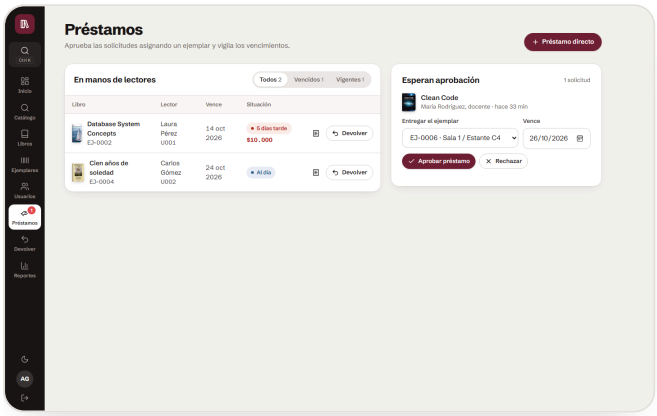


Búsqueda por título, autor, ISBN, categoría y "solo disponibles". Las portadas reales vienen de Open Library por ISBN; si no hay, se genera una.

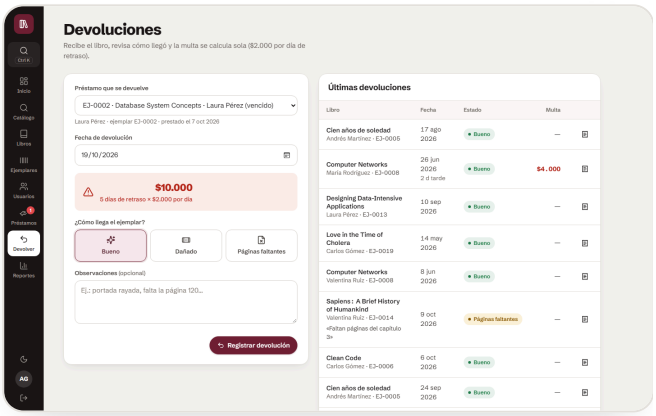


Detalle: datos, disponibilidad con ubicación, ejemplares e historial de préstamos (este último solo lo ve el bibliotecario).

Préstamos y devoluciones

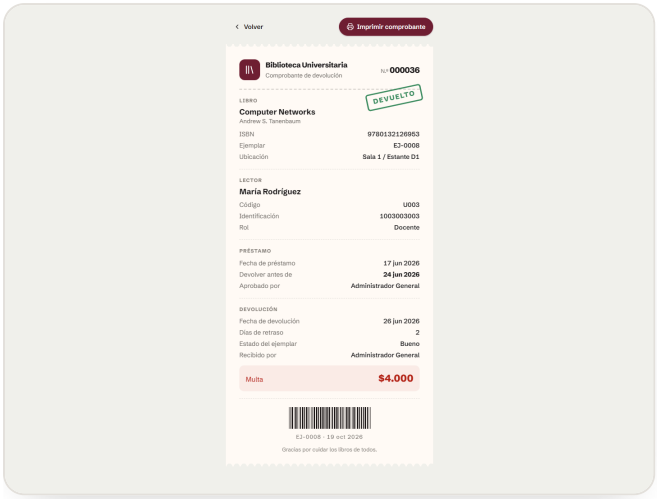


A la izquierda los préstamos activos con su situación y multa estimada; a la derecha las solicitudes para aprobar (eligiendo ejemplar y vencimiento) o rechazar.

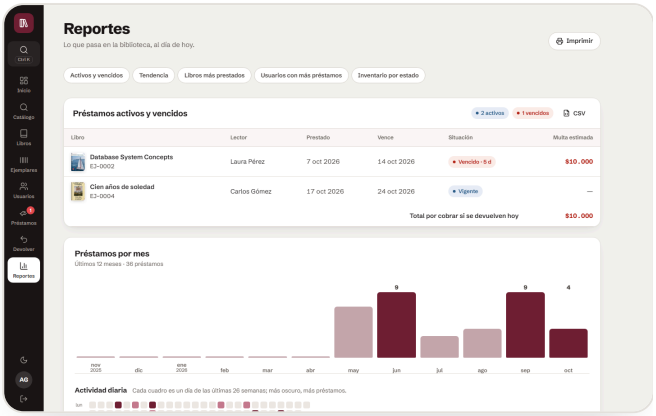


Se elige el préstamo y la fecha; la multa se calcula en vivo. Se marca cómo llega el ejemplar: bueno, dañado o páginas faltantes.

Comprobante y reportes

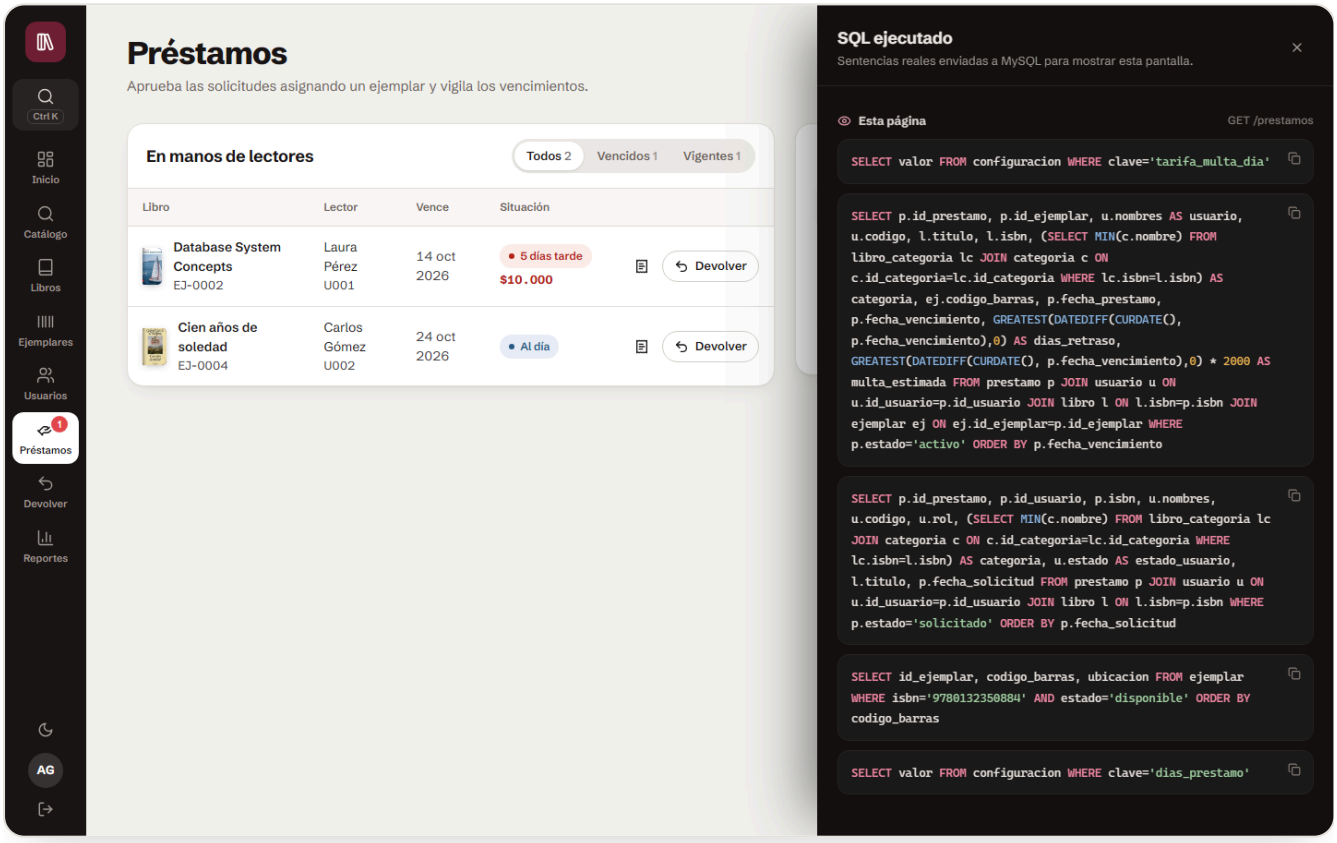


Comprobante imprimible con sello "Prestado" o "Devuelto", datos del lector, fechas, quién aprobó, multa y un código de barras decorativo.



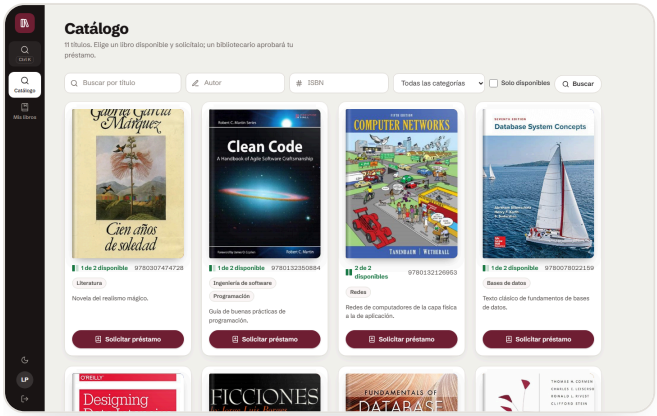
Reportes: activos y vencidos con total por cobrar, préstamos por mes, mapa de calor diario, libros y usuarios con más préstamos e inventario. Cada uno se exporta a CSV.

Ver SQL

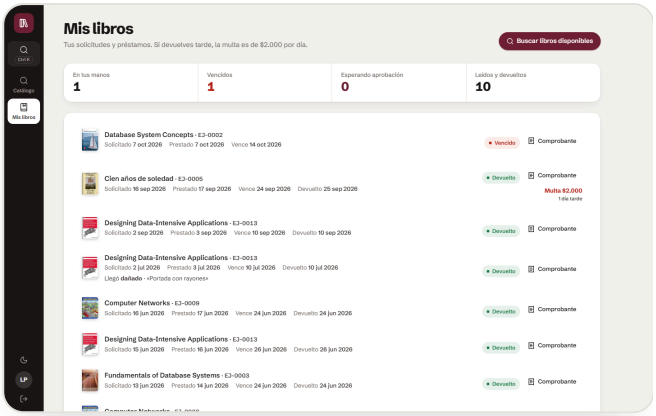


El botón "Ver SQL" abre este panel con las consultas exactas que se enviaron a MySQL para mostrar la pantalla. Después de una acción (aprobar, devolver) también muestra la transacción completa con su COMMIT.

La vista del lector



El lector ve el catálogo con el botón "Solicitar préstamo" (desactivado si no hay ejemplares).



"Mis libros": lo que tiene en sus manos, lo vencido, lo pendiente de aprobación y su historial con multas.

Otras funciones de la interfaz

- **Buscador rápido (Ctrl+K o la tecla /):** busca libros con su portada, salta a cualquier sección y lanza búsquedas por autor, ISBN o usuario.
- **Modo oscuro:** botón luna/sol en el menú; recuerda tu preferencia.
- **Celular:** el menú pasa abajo y las tablas se convierten en tarjetas.
- **Animación de bienvenida:** al iniciar sesión, un libro se abre y te saluda con tu nombre (se salta con un clic).

PARTE 8

Las reglas del negocio

Estas son las reglas que la aplicación hace cumplir. Todas viven en `db.py`.

Al solicitar un préstamo (lector)

1. El usuario debe estar **activo** (los bloqueados no pueden pedir).
2. No puede tener ya una solicitud o préstamo vigente **del mismo libro**.
3. No puede superar el **máximo de préstamos simultáneos** (3, configurable en la tabla `configuracion`).
4. El libro debe tener al menos **un ejemplar disponible**.

Si todo se cumple, se crea un préstamo con estado `solicitado`.

Al aprobar o prestar (bibliotecario)

1. El usuario sigue activo y el ejemplar sigue disponible (se revisa otra vez, con candado `FOR UPDATE`).
2. El ejemplar corresponde al libro solicitado.
3. La fecha de vencimiento es posterior a la del préstamo (por defecto, 7 días después).
4. La solicitud no fue atendida ya por otra persona.

Al devolver: la multa

La fórmula

días de retraso = días entre la fecha de vencimiento y la fecha de devolución (si es negativo, se cuenta 0).
multa = días de retraso × tarifa por día (**$2.000**, guardada en `configuracion.tarifa_multa_dia`).

Ejemplo: vencía el 14 de octubre y se devuelve el 19 → 5 días × $2.000 = **$10.000**.

Además: la fecha de devolución no puede ser futura ni anterior al préstamo; el préstamo pasa a `devuelto`; el ejemplar vuelve a `disponible`; y se guardan la condición (bueno, dañado, páginas faltantes) y las observaciones. Todo dentro de una transacción.

Al eliminar

La base de datos protege la historia: no se puede borrar un libro con ejemplares, un ejemplar con préstamos ni un usuario con préstamos (las claves foráneas usan `ON DELETE RESTRICT`). La app lo explica con un mensaje y sugiere la alternativa: marcar el ejemplar como perdido o bloquear al usuario.

¿Por qué la tarifa está en una tabla y no en el código?

Porque el enunciado pide "definir una tarifa de multa por día" y es un dato que la biblioteca podría cambiar. Guardándola en `configuracion`, se cambia con un `UPDATE` sin tocar ni reiniciar el programa. Lo mismo pasa con los días de préstamo y el máximo de préstamos.

PARTE 9

Seguridad

Riesgo	Cómo lo evitamos
Robo de contraseñas	Las claves del personal no se guardan en texto: se guarda un **hash PBKDF2-SHA256** con 100.000 iteraciones y una "sal" aleatoria. Ni siquiera quien vea la base puede saber la clave original.
Inyección SQL	Nunca se pega texto del usuario dentro del SQL. Se usan **consultas parametrizadas** (`WHERE codigo=%s`) y PyMySQL escapa los valores.
Entrar donde no debes	El decorador `requiere()` revisa la sesión y el rol en cada ruta. Un lector no puede ver la gestión ni los comprobantes de otras personas.
Sesiones falsificadas	La sesión se guarda en una cookie firmada con `SECRET_KEY`; si alguien la modifica, Flask la rechaza.
Contraseñas en GitHub	`config.toml` y `config.nube.toml` están en `.gitignore`. En la nube las claves van como variables de entorno en Render, nunca en el código.
Datos inconsistentes	Claves foráneas, UNIQUE, ENUM y transacciones hacen que la base rechace datos inválidos aunque alguien se salte la aplicación.

Ojo

La identificación del lector funciona como su "clave" de ingreso, como pide el enunciado con datos mínimos. En un sistema real se pediría una contraseña aparte. La cuenta `admin / admin123` es solo de demostración.

PARTE 10

Ejecutarla en tu computador

Lo que necesitas y los pasos exactos, en Windows.

Necesitas

- **Python 3.13** (ya está instalado en tu PC).
- **MySQL 8** encendido (el servicio `MySQL80`) y MySQL Workbench opcional.
- La carpeta del proyecto: `C:\Users\sebas\Downloads\Biblioteca\Biblioteca`.

Primera vez

1. **Crear la base de datos.** Abre una terminal (PowerShell) en la carpeta del proyecto y ejecuta `.\.venv\Scripts\python.exe cargar_bd.py`. También puedes abrir `schema.sql` en MySQL Workbench y ejecutarlo completo.
2. **Revisar la configuración.** `config.toml` debe tener tu usuario y clave de MySQL. Si no existe, copia `config.example.toml` con ese nombre y llénalo.
3. **Instalar librerías** (solo si creas un entorno nuevo): `py -3.13 -m venv .venv` y luego `.\.venv\Scripts\pip install -r requirements.txt`.

Cada vez que la quieras usar

```
cd C:\Users\sebas\Downloads\Biblioteca\Biblioteca
.\.venv\Scripts\activate
python app.py
```

Luego abre **http://127.0.0.1:5000** en el navegador. Para apagarla, pulsa `Ctrl + C` en la terminal.

> **Consejo**
>
> Si haces pruebas y quieres volver a los datos originales antes de la sustentación, ejecuta de nuevo `cargar_bd.py`: borra la base `biblioteca` y la crea como nueva (no toca tus otras bases).

PARTE 11

Publicada en internet

La aplicación funciona en **https://biblioteca-93xx.onrender.com** sin necesidad de tu computador. Son tres servicios trabajando juntos:

G

GitHub

Guarda el código. Cada cambio se sube con `git push`.

R

Render

Descarga el código de GitHub, instala las librerías y ejecuta Flask con `gunicorn`. Plan gratis.

A

Aiven

Aloja la base de datos MySQL en la nube (plan gratis, servidor en San Francisco). La conexión va cifrada (SSL).

Cómo se conectan

Render no tiene `config.toml` (no se sube a GitHub). En su lugar, en el panel de Render están las **variables de entorno** `MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_DATABASE` y `MYSQL_SSL`. La función `load_config()` de `db.py` primero busca esas variables; si no existen, usa `config.toml`. Así el mismo código funciona en tu PC y en la nube.

Ojo: la app "se duerme"

En el plan gratis, si nadie la visita en 15 minutos, Render la apaga. Se vuelve a prender sola al abrir el link, pero esa primera carga tarda entre 30 y 50 segundos. **Antes de la sustentación, ábrela unos minutos antes.** Los datos no se pierden, porque viven en Aiven.

Actualizar la versión publicada

Cuando se cambia el código: se hace `git push` a GitHub y en Render se pulsa **Manual Deploy → Deploy latest commit** (o se activa **Auto-Deploy: On Commit** para que sea automático).

PARTE 12

Preparación para la sustentación

Preguntas que probablemente te hará el profesor, con respuestas cortas que puedes decir con tus palabras.

P. ¿Qué tecnologías usaron y por qué?

Python con Flask, que es una de las opciones del enunciado, MySQL 8 como motor relacional y PyMySQL como driver. Flask es ligero y deja ver claramente cada ruta y cada consulta SQL.

P. ¿Por qué existe la tabla libro_autor?

Porque libro y autor tienen una relación muchos a muchos. Una tabla puente con PK compuesta (isbn, id_autor) guarda cada pareja y evita repetir datos.

P. ¿Cuál es la diferencia entre libro y ejemplar?

El libro es la obra (un ISBN, un título); el ejemplar es cada copia física con su código de barras, ubicación y estado. Se presta el ejemplar, no el libro.

P. ¿Cómo validan que el usuario esté activo y el ejemplar disponible?

En db._validar_y_prestar, dentro de una transacción, con SELECT ... FOR UPDATE sobre el usuario y el ejemplar. Si algo no cumple se hace ROLLBACK.

P. ¿Qué pasa si dos bibliotecarios prestan el mismo ejemplar a la vez?

El FOR UPDATE bloquea la fila: el segundo espera, ve que ya está prestado y recibe el error "El ejemplar ya no está disponible".

P. ¿Cómo se calcula la multa?

Días de retraso (devolución menos vencimiento, mínimo 0) por la tarifa diaria de la tabla configuracion, que es $2.000.

P. ¿Dónde se aprueba el préstamo?

El lector crea una solicitud (estado solicitado). El bibliotecario la ve en Préstamos, asigna un ejemplar y la aprueba (estado activo), o la rechaza. Se guarda quién la aprobó en prestamo.id_personal.

P. ¿Por qué id_ejemplar acepta NULL en préstamo?

Porque al solicitar todavía no se sabe qué copia se entregará; se asigna al aprobar.

P. ¿Qué es una clave foránea? Muestre una.

Una columna que referencia la PK de otra tabla. Por ejemplo prestamo.id_usuario referencia usuario.id_usuario; MySQL no deja crear un préstamo de un usuario que no existe.

P. ¿Por qué la devolución es una tabla aparte?

Porque solo existe cuando el préstamo termina (relación 1 a 0..1) y tiene datos propios: retraso, multa, condición, observaciones y quién la recibió.

P. ¿Cómo sale el reporte de libros más prestados?

Con un SELECT que une prestamo y libro, filtra por estado y rango de fechas, agrupa por libro (GROUP BY) y ordena por COUNT(*) de mayor a menor.

P. ¿Cómo se protegen las contraseñas?

Se guardan como hash PBKDF2-SHA256 con sal; al ingresar se calcula el hash de lo escrito y se compara.

P. ¿Cómo evitan la inyección SQL?

Todas las consultas son parametrizadas (%s); nunca se concatena texto del usuario dentro del SQL.

P. ¿Qué pasa si intento borrar un usuario con préstamos?

La FK con ON DELETE RESTRICT lo impide; la app muestra el mensaje y sugiere bloquearlo.

P. ¿Pueden mostrar el SQL que se ejecuta?

Sí, con el botón Ver SQL de cualquier pantalla.

Guion de demostración (5 minutos)

1. Abre el link (o tu PC) unos minutos antes. Muestra el login y entra como lector `U001`.
2. Busca un libro en el catálogo y solicita el préstamo. Muestra "Mis libros".
3. Cierra sesión, entra como `admin`, ve a Préstamos, aprueba la solicitud y abre **Ver SQL**.
4. Ve a Devolver, registra la devolución vencida de Laura y muestra la multa de $10.000 y el comprobante.
5. Termina en Reportes y muestra los diagramas MER y relacional.

ANEXO

Glosario

Término	Significado
Backend / Frontend	Backend es lo que corre en el servidor (Python, MySQL). Frontend es lo que corre en el navegador (HTML, CSS, JS).
CRUD	Crear, Leer, Actualizar y Borrar: las cuatro operaciones básicas sobre datos.
Decorador	En Python, una línea con `@` encima de una función que le agrega comportamiento (por ejemplo `@requiere` agrega el control de permisos).
Deploy (despliegue)	Publicar la aplicación en un servidor de internet.
Driver	Librería que permite a un lenguaje hablar con un motor de base de datos (PyMySQL).
Endpoint / ruta	Una dirección de la app (`/reportes`) asociada a una función.
Framework	Un conjunto de herramientas que resuelve lo repetitivo para que te concentres en tu lógica (Flask).
Hash	Una "huella" irreversible de un texto. Sirve para guardar contraseñas sin guardarlas.
Índice	Una estructura que acelera las búsquedas en una columna, como el índice de un libro.
JOIN	Unir filas de dos tablas por una columna en común (por ejemplo, préstamo con usuario).
Plantilla (template)	Un HTML con huecos que Flask llena con datos.
Repositorio	La carpeta del proyecto con todo su historial en Git.
Sesión	La forma en que la app recuerda quién inició sesión entre una página y otra.
Variable de entorno	Un valor de configuración que se le da al programa desde afuera (usado en Render para las credenciales).

Ficha rápida

App en internet	**https://biblioteca-93xx.onrender.com**
App en tu PC	http://127.0.0.1:5000 (después de `python app.py`)
Código	https://github.com/lcorb10/Biblioteca
Bibliotecario	`admin` / `admin123`
Lector de prueba	código `U001` / identificación `1001001001`
Tarifa de multa	$2.000 por día · 7 días de préstamo · máximo 3 préstamos
Entregables	`docs/MER.png` , `docs/Modelo_Relacional.png` , `schema.sql` , código de la app